Python Mini Project - Complete Assignment Solutions

Q1 - Student Result Analyzer

```
def analyze_result(name, roll, marks):
    total = sum(marks)
    average = total / len(marks)

    if average >= 90:
        grade = "A"
    elif average >= 75:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 40:
        grade = "D"
    else:
        grade = "Fail"

    print(f"Student: {name} (Roll: {roll})")
    print(f"Total: {total}, Average: {average}")
    print(f"Grade: {grade}")

    below_40 = []

    for i in range(len(marks)):
        if marks[i] < 40:
            below_40.append(f"Subject {i + 1}")

    if below_40:
        print("Subjects below 40:", ", ".join(below_40))
    else:
        print("No subjects below 40")


# Sample Input
name = "Aarav"
roll = 101
marks = [88.5, 35.0, 76.0, 92.5, 48.0]

analyze_result(name, roll, marks)
```

Q2 - Library Book Management System

```
def add_book(catalog, book_id, title, author, year):
    catalog[book_id] = (title, author, year)


def borrow_book(catalog, borrowed_books, book_id):
    if book_id in catalog and book_id not in borrowed_books:
        borrowed_books.append(book_id)
        print(f"Book {book_id} borrowed successfully.")
    else:
        print(f"Book {book_id} cannot be borrowed.")


def return_book(borrowed_books, book_id):
    if book_id in borrowed_books:
        borrowed_books.remove(book_id)
        print(f"Book {book_id} returned successfully.")
    else:
        print(f"Book {book_id} was not borrowed.")


def register_member(members, member_id):
    members.add(member_id)
def show_available(catalog, borrowed_books):
    print("\nAvailable Books:")
```

```
    for book_id, details in catalog.items():
        if book_id not in borrowed_books:
            print(f"{book_id}: {details}")


def main():
    catalog = {}
    borrowed_books = []
    members = set()

    # Add books
    add_book(catalog, 1, "Python Basics", "John Smith", 2020)
    add_book(catalog, 2, "Data Science", "Alice Brown", 2021)
    add_book(catalog, 3, "AI Fundamentals", "David Lee", 2022)
    add_book(catalog, 4, "Cyber Security", "Emma Wilson", 2023)

    # Register members
    register_member(members, 101)
    register_member(members, 102)
    register_member(members, 103)
    register_member(members, 101)  # Duplicate

    print("Registered Members:", members)

    # Borrow books
    borrow_book(catalog, borrowed_books, 1)
    borrow_book(catalog, borrowed_books, 3)

    print("Borrowed Books:", borrowed_books)

    # Return one book
    return_book(borrowed_books, 1)

    print("Borrowed Books after return:", borrowed_books)

    # Show available books
    show_available(catalog, borrowed_books)


main()
```

Q3 - Shopping Cart with Mutable Default Argument

```
# Part A - Spot the Bug

def add_item(item, cart=[]):
    cart.append(item)
    return cart


print(add_item("apple"))
print(add_item("banana"))
print(add_item("milk", cart=["bread"]))
print(add_item("eggs"))

# Explanation:
# The default list is shared between function calls.
# So apple, banana, and eggs stay in the same list.


# Part B - Fix It

def add_item_fixed(item, cart=None):
    if cart is None:
        cart = []

    cart.append(item)
    return cart


print(add_item_fixed("apple"))
print(add_item_fixed("banana"))


# Part C - Shopping Cart Program
```

```
def create_cart(owner, discount=0):
    return {
        "owner": owner,
        "items": [],
        "discount": discount
    }


def add_to_cart(cart, name, price, qty=1):
    item = {
        "name": name,
        "price": price,
        "qty": qty
    }

    cart["items"].append(item)


def update_price(price_tuple, new_price):
    # Tuples are immutable, so modifying them raises TypeError
    price_tuple[0] = new_price


def calculate_total(cart):
    total = 0

    for item in cart["items"]:
        total += item["price"] * item["qty"]

    discount_amount = total * (cart["discount"] / 100)
    final_total = total - discount_amount

    return final_total


# Customer 1
cart1 = create_cart("Aarav", 10)
add_to_cart(cart1, "Laptop", 50000, 1)
add_to_cart(cart1, "Mouse", 1000, 2)

# Customer 2
cart2 = create_cart("Diya", 5)
add_to_cart(cart2, "Phone", 30000, 1)

print("\nCart 1:", cart1)
print("Total Cart 1:", calculate_total(cart1))

print("\nCart 2:", cart2)
print("Total Cart 2:", calculate_total(cart2))


# Tuple immutability example
try:
    prices = (100, 200, 300)
    update_price(prices, 500)
except TypeError as e:
    print("\nError:", e)


# Discussion Points
# 1. discount=0 is safe because integers are immutable.
#    cart=[] is dangerous because lists are mutable and shared.

# 2. Rebinding means assigning a new object.
#    Mutating means changing the existing object.

# 3. Mutable: list, dict, set
#    Immutable: tuple, str, int

# 4. Yes, changes reflect outside because lists are mutable
#    and functions receive references to the same object.
```